

Guia Completo — Governança + Bootstrap do Microserviço Users (Aurora)

Para alunos: este guia reúne **tudo que já foi feito** e **todo o embasamento** para que você entenda e consiga aplicar. Cobre: governança do repositório (proteção da `main`, CI, squash), CODEOWNERS/PR template/checklist, e a implementação **até o item 1.1 (criar o projeto)** no **repo simples**. No final há um **log de problemas & soluções** e um **passo-a-passo de teste**.

Sumário

1. [Visão Geral e Objetivos](#)
 2. [Governança do Repositório](#)
 3. 2.1 [Proteção da `main`: o que cada opção garante](#)
 4. 2.2 [Configurar/Verificar via CLI \(gh api\)](#)
 5. 2.3 [Botão de Merge \(Squash\) e Histórico Linear](#)
 6. 2.4 [CODEOWNERS, PR Template e Checklist de Auditoria](#)
 7. [Implementação até 1.1 — Criar o projeto \(repo simples\)](#)
 8. 3.1 [Estratégia de scaffold \(sem conflitos\)](#)
 9. 3.2 [Instalação e lockfile](#)
 10. 3.3 [Ajustes de Lint necessários](#)
 11. 3.4 [CI para repo simples \(e preparada para monorepo\)](#)
 12. 3.5 [Porta da aplicação: 3001](#)
 13. 3.6 [Arquivos criados \(o que é e para que serve\)](#)
 14. [Conceitos-chave explicados](#)
 15. [Problemas reais que aconteceram & como resolvemos](#)
 16. [Como testar \(local e CI\)](#)
 17. [Checklist do que já está pronto](#)
 18. [Próximos passos \(pendentes\)](#)
-

Visão Geral e Objetivos

- Começar a plataforma **Aurora** pelo microserviço **Users**.
 - Aplicar práticas de **governança** desde o início: PR obrigatório, CI com `lint/build/test`, proteção da `main`, histórico linear (**Squash and merge**), evidências de auditoria.
 - **Repo simples** (um serviço no *root*). O pipeline já está preparado para uma futura migração para **monorepo**.
-

Governança do Repositório

Proteção da `main`: o que cada opção garante

Opção	O que garante / Risco mitigado	Observação prática
Require a pull request before merging	Todo código entra via PR → histórico claro, revisão e CI	Mesmo trabalhando solo, PR mantém rastreabilidade
Require status checks to pass (<code>lint</code> , <code>build</code> , <code>test</code>)	Evita quebrar a <code>main</code> ; gate de qualidade	Os 3 jobs existem na CI
Require branches to be up to date (<code>strict:true</code>)	PR precisa revalidar contra a <code>main</code> atual	Evita merges “velhos”
Enforce admins	Ninguém pula as regras por acidente	Torna o padrão sólido
Disallow force push / deletions	Protege o histórico e a branch principal	Segurança do repositório
Approvals exigidas	(Cenário solo: 0). Em equipes: 1..2	Em times, some com Code Owners
Conversation resolution	Não deixa mergear com comentários abertos	Melhora qualidade da revisão
Linear history	Histórico sem <i>merge commits</i>	Combine com Squash

Cenário **solo** usado aqui: PR obrigatório, **0 aprovações exigidas**, checks `lint/build/test`, `strict`, conversas resolvidas, linear history, sem force push/deleção e **enforce admins**.

Configurar/Verificar via CLI (gh api)

Aplicar/ajustar (solo):

```
gh api -X PUT -H "Accept: application/vnd.github+json"
  repos/:owner/:repo/branches/main/protection
  --input - <<'JSON'
{
  "required_status_checks": { "strict": true, "contexts":
["lint","build","test"] },
  "enforce_admins": true,
  "required_pull_request_reviews": {
    "required_approving_review_count": 0,
    "dismiss_stale_reviews": false,
    "require_last_push_approval": false,
    "require_code_owner_reviews": false
  },
  "required_conversation_resolution": true,
  "required_linear_history": true,
```

```
"allow_force_pushes": false,
"allow_deletions": false,
"restrictions": null
}
JSON
```

Checar estado:

```
gh api repos/:owner/:repo/branches/main/protection | jq '{
  strict: .required_status_checks.strict,
  contexts: .required_status_checks.contexts,
  reviews: .required_pull_request_reviews,
  enforce_admins: .enforce_admins.enabled,
  linear: .required_linear_history.enabled,
  conversations: .required_conversation_resolution.enabled
}'
```

Botão de Merge (Squash) e Histórico Linear

- **Por que:** com `required_linear_history=true`, merges com *merge commit* são bloqueados; deixar **só Squash** simplifica o fluxo: **1 PR → 1 commit**.
- **UI:** desative *Allow merge commits*; ative *Allow squash merging*; (opcional) desative *Allow rebase merging*; (opcional) ative *Delete branch on merge*.
- **CLI** (rodar dentro do repo):

```
gh repo edit
  --enable-merge-commit=false
  --enable-rebase-merge=false
  --enable-squash-merge=true
  --delete-branch-on-merge
```

Verificar:

```
gh api repos/:owner/:repo | jq '{
  allow_merge_commit: .allow_merge_commit,
  allow_rebase_merge: .allow_rebase_merge,
  allow_squash_merge: .allow_squash_merge,
  delete_branch_on_merge: .delete_branch_on_merge
}'
```

CODEOWNERS, PR Template e Checklist de Auditoria

- **CODEOWNERS** (opcional no modo solo; útil para times):
- `/.github/CODEOWNERS`
- Exemplo simples:

```
/src/**                                @org/time-devs @aluno1 @aluno2
/test/**                               @org/time-devs
```

```
/.github/** @org/devops
/* @org/time-devs
```

- **PR Template** (`.github/pull_request_template.md`): padroniza contexto, evidências e validações.
- **Checklist no README**: bloco “Auditoria — Proteção da `main`” com itens marcados e **links/prints** de evidência.

Implementação até 1.1 — Criar o projeto (repo simples)

3.1 Estratégia de scaffold (sem conflitos)

Problema comum: `nest new .` em um diretório que já tem `README.md` / `.github/` → conflito e aborta.

Solução aplicada: gerar em **pasta temporária** e **copiar** para o repo, preservando a governança:

```
mkdir -p /tmp/nest-skel && cd /tmp/nest-skel
nest new aurora-users --package-manager npm --strict --skip-git --skip-install
rsync -av /tmp/nest-skel/aurora-users/
"/caminho/do/repo/"
--exclude '.git' --exclude '.github' --exclude 'README.md'
```

Depois, **commit** em uma branch (ex.: `feat/nest-skeleton`) e abrir **PR**.

3.2 Instalação e lockfile

- Rode `npm install` para gerar o `package-lock.json` e **versione** esse arquivo.
- A CI usa `npm ci` (mais rápido e reprodutível), que **exige** lockfile.

3.3 Ajustes de Lint necessários

- **Regra** `no-floating-promises` (promessas “boiando”) → em `src/main.ts`:

```
async function bootstrap() { /* ... */ }
void bootstrap();
```

- **Família** `no-unsafe-*` (tipos inseguros) no E2E → em `test/app.e2e-spec.ts`:

```
import request from 'supertest';
await request(app.getHttpServer()).get('/').expect(200);
```

3.4 CI para repo simples (e preparada para monorepo)

- Workflow `.github/workflows/ci.yml` com **3 jobs**: `lint`, `build`, `test`.

- Em **repo simples**, use `npm run ...`. Para **monorepo**, pode-se detectar `"workspaces"` e usar `npm -ws run ...`.
- Mantemos `npm ci` nos jobs (depende do `package-lock.json`).

3.5 Porta da aplicação: 3001

- Em `src/main.ts` (recomendado via variável de ambiente):

```
const port = parseInt(process.env.PORT ?? '3001', 10);
await app.listen(port);
```

- Rodar local: `PORT=3001 npm run start:dev` → `http://localhost:3001/`

3.6 Arquivos criados (o que é e para que serve)

- `src/main.ts` — *bootstrap* do servidor Nest; define porta e middlewares globais.
- `src/app.module.ts` — módulo raiz; registra controllers/services e outros módulos.
- `src/app.controller.ts` / `src/app.service.ts` — exemplo inicial da rota `GET /`.
- `test/app.e2e-spec.ts` — teste E2E mínimo de `GET /` com `supertest`.
- `package.json` — scripts `start`, `start:dev`, `build`, `lint`, `test`, `test:e2e` usados localmente e na CI.
- `package-lock.json` — lockfile **necessário** para `npm ci` (CI estável/reprodutível).
- `nest-cli.json` — configuração do Nest CLI (sourceRoot, entry, schematics).
- `tsconfig.json` / `tsconfig.build.json` — configs TypeScript para desenvolvimento e build.
- `.eslintrc.*` e `.prettierrc*` — regras de qualidade (ESLint) e formatação (Prettier).
- `.github/workflows/ci.yml` — pipeline com `lint/build/test` (checks exigidos pela proteção da `main`).

Conceitos-chave explicados

- `no-floating-promises`: toda Promise deve ser aguardada (`await`), tratada (`.catch`) ou marcada como ignorada (`void`). Evita rejeições silenciosas e *race conditions*.
- **Família** `no-unsafe-*`: impede chamar/acessar/retornar valores `any/unknown` sem refino. Melhor tipagem = menos bugs.
- **Teste E2E**: testa o sistema como **caixa-preta** via HTTP, percorrendo o fluxo completo (controller → service → camadas internas) e verificando resposta. Complementa unit/integration.
- `jest.config.ts`: configuração global do Jest (transform TS, descoberta de testes, cobertura, mapeamentos de módulo).
- `test/jest-e2e.json`: configuração dedicada aos E2E (padrões de arquivo, `rootDir`, possíveis *setups* específicos).
- **Scripts do** `package.json`: contrato com a CI e padrão de execução local (`start`, `start:dev`, `build`, `lint`, `test`, `test:e2e`).
- `nest-cli.json`: guia do Nest CLI para gerar/compilar (paths, entry file, schematics).
- `tsconfig*.json`: base de TypeScript para dev vs. build, permitindo exclusões e ajustes apropriados ao empacotamento.
- **ESLint × Prettier**: qualidade da lógica vs. formatação do código; use ambos em harmonia.

Problemas reais que aconteceram & como resolvemos

1. **zsh:** `[]` em `contexts[]` (Branch Protection via `-F`): o shell tenta expandir `[]` → erro.
Solução: usar JSON no `gh api` (robusto) ou escapar/aspas.
2. **Campos aninhados ignorados** com `-F` (form-data): alguns subcampos não aplicaram.
Solução: enviar **corpo JSON** completo no `PUT` de proteção.
3. **Só status checks aplicaram;** reviews e extras ficaram `false`: consequência do ponto anterior.
Solução: `gh api` com JSON consolidado.
4. **Merge button:** flags `--disable-*` não existem. **Solução:** usar `--enable-merge-commit=false` etc., ou PATCH REST com `allow_*` booleans.
5. **404 em** `owner/:repo`: placeholder literal. **Solução:** usar `owner/repo` real ou rodar o comando **dentro** do repo.
6. **PR "Closed" sem merge** com `CODEOWNERS`: sem merge, o arquivo não entra na `main` e não tem efeito. **Solução:** reabrir e **Squash and merge**.
7. `nest new` **conflitando com** `README.md`: diretório não vazio. **Solução:** gerar em `/tmp` e copiar com `rsync` preservando `.github/README`.
8. **Falha no** `npm install` **do Nest CLI:** etapa automática pode falhar; **Solução:** gerar com `--skip-install` e instalar manualmente com logs.
9. **CI falhando em** `npm ci`: faltava `package-lock.json`. **Solução:** versionar o lockfile.
10. **Erros de lint:** `no-floating-promises` (bootstrap) e `no-unsafe-*` (E2E). **Solução:** `void bootstrap()` e `supertest` com `default import + await`.
11. **Workflow usando** `npm -ws run ...` em repo simples: não há `workspaces`. **Solução:** detectar `"workspaces"` e usar `npm run ...` quando não houver.
12. **Lockfile com dono** `root` (ambiente local): pode gerar atritos em `node_modules`. **Solução:** normalizar permissões do diretório do repo.

Como testar (local e CI)

Local

```
npm ci
npm run lint
npm run build
npm test
npm run test:e2e
PORT=3001 npm run start:dev
# abrir http://localhost:3001/
```

CI (no PR) - Verificar os 3 jobs: **lint** → **build** → **test**. Todos devem ficar **verdes**. - Anexar no PR as **evidências** (links dos jobs e print da proteção da `main`).

Checklist do que já está pronto

- [x] Proteção da `main` aplicada (PR, checks `lint/build/test`, strict, conversas resolvidas, linear history, enforce admins, sem force push/deleção).
- [x] Botão de merge ajustado (**Squash**).

- [x] PR template e checklist de auditoria no README.
 - [x] Skeleton NestJS criado no root sem conflitos.
 - [x] Lockfile versionado para `npm ci`.
 - [x] Lint ajustado (`main.ts` e E2E).
 - [x] CI rodando os 3 jobs para repo simples.
 - [x] App pronto para rodar na **porta 3001**.
-

Próximos passos (pendentes)

- Banco **PostgreSQL** + TypeORM (datasource + Docker Compose + `.env.example`).
- **Migrations** (sem `synchronize:true`).
- **CRUD de Users** (entidade, DTOs, service, controller) com `bcrypt`, e-mail único e sem expor `passwordHash`.
- Job **test** da CI com Postgres como **service**.
- **Swagger** (`/api/docs`), **ValidationPipe** global e rota `/health`.

Com este guia, você tem a base completa para continuar a implementar o microserviço **Users** com governança sólida e pipeline confiável. A partir daqui, é iterar sobre banco, migrations e CRUD, mantendo os mesmos padrões de qualidade.